LLM-Sentry: A Large Language Model Framework for Detecting Malicious Packages and Dependency Poisoning in Software Supply Chains

Allan Douglas Costa
Federal Rural University of the Amazon (UFRA)
Institute of Cyberspace (ICIBE)
Belém, Brazil
allan.costa@ufra.edu.br
https://orcid.org/0000-0002-7068-8889

Abstract—Software supply chain attacks via public package registries such as npm and PyPI have grown exponentially, with over 1.2 million malicious packages catalogued through 2025. Existing rule-based and static-analysis defenses fail to detect novel obfuscation patterns and zero-day poisoning campaigns. This paper presents LLM-Sentry, a multi-stage detection framework that combines fine-tuned large language models (LLMs) with static metadata analysis and behavioral sequence modeling to identify malicious packages and dependency poisoning across heterogeneous ecosystems. LLM-Sentry introduces a novel composite metric, the Package Risk Confidence Score (PRCS), which integrates semantic code embeddings, provenance metadata entropy, and behavioral API call deviation to quantify package-level threat probability. We evaluate LLM-Sentry on the publicly available PyPI Malware Dataset and the MalwareBazaar-OSS benchmark, comprising 18,942 labeled packages. Experimental results demonstrate that LLM-Sentry achieves a detection F1-score of 0.961, outperforming baseline approaches including MalOSS, DONAPI, and traditional static-analysis tools by 6.3 to 14.8 percentage points. The framework detects typosquatting, dependency confusion, and code-injection attacks with false-positive rates below 1.2%. All code, datasets, and evaluation scripts are publicly available at https://github.com/ufxa/Software-Supply-Chain-Security.

Index Terms—software supply chain security, malicious package detection, large language models, dependency poisoning, npm, PyPI, DevSecOps, static analysis, LLM-based detection, PRCS metric

I. Introduction

The modern software development lifecycle depends critically on open-source package registries. npm hosts over 2.5 million packages and PyPI exceeds 600,000, while Maven Central and NuGet collectively serve billions of download requests monthly [1]. This dependency ecosystem accelerates development but simultaneously expands the software supply chain attack surface to an unprecedented scale.

A software supply chain attack occurs when an adversary compromises a dependency that is subsequently integrated into downstream applications [2]. The 2020 SolarWinds SUNBURST incident demonstrated how compromising a single build system could affect 18,000 organizations [3]. The 2024 XZ Utils backdoor (CVE-2024-3094) revealed that a determined social engineering campaign lasting years could insert a CVSS 10.0 remote-code-execution vulnerability into a critical open-source library [4]. Sonatype's 2025 State of the Software Supply Chain report identifies over 454,600 new malicious packages published in a single year, a 156% increase over 2023 [1].

Existing defenses operate in three paradigms. Rule-based approaches flag known malicious indicators, such as obfuscated install scripts or known attacker domains, but miss novel campaigns [5]. Static analysis tools parse code structure and abstract syntax trees (ASTs) but struggle with heavily obfuscated JavaScript and Python [6]. Machine learning classifiers, including graph neural networks and behavioral models, have shown promise but exhibit poor cross-ecosystem generalization [7]. None of these paradigms leverage the semantic understanding capacity of modern LLMs.

Recent work demonstrates that LLMs, particularly GPT-4 and fine-tuned Code-specialized models, exhibit strong capabilities in code comprehension, vulnerability detection, and anomaly reasoning [8], [9]. However, their systematic application to supply chain malware detection remains an open problem, with only preliminary exploratory studies available [?], [10].

Research Gaps. No prior work provides: (1) an end-to-end LLM-based framework for cross-ecosystem malicious package detection; (2) a composite metric that unifies semantic, provenance, and behavioral signals; or (3) a reproducible evaluation on a standardized public benchmark with confidence intervals across multiple attack types.

Contributions. This paper makes the following contributions:

1) LLM-Sentry framework: A six-stage pipeline integrating LLM semantic analysis, metadata entropy scoring, and API behavioral modeling for supply chain threat detection (Section III).
2) Package Risk Confidence Score (PRCS): A novel composite metric with formal definition, combining three independently computed sub-scores into a unified risk quantifier (Section IV).

3) Empirical evaluation: Comprehensive experiments on 18,942 labeled packages demonstrating state-of-the-art performance, with 95% confidence intervals, ablation studies, and cross-ecosystem generalization tests (Section VI).
4) Reproducibility: All code, datasets, and evaluation pipelines are released publicly at https://github.com/ufxa/Software-Supply-Chain-Security.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the LLM-Sentry architecture. Section IV formalizes the PRCS metric. Section V details the experimental setup. Section VI presents results and discussion. Section VIII provides a security analysis. Section IX addresses compliance and ethical aspects. Section X concludes.

II. Related Work

A. Software Supply Chain Attacks

Software supply chain attacks exploit the trust relationships inherent in package dependency resolution. Ohm et al. categorize the attack surface into six vectors: open-source package uploads, compromised maintainer accounts, dependency confusion, build-system injection, source-code repository compromise, and typosquatting [2]. The dependency confusion attack, first demonstrated by Birsan in 2021, exploits version resolution priority in private registries and has been replicated against major enterprises [11]. ConfuGuard [12] detected 630 real package-confusion attacks in production registries, reducing the false-positive rate from 80% to 28% through registry-level signature analysis.

Recent reports indicate that typosquatted packages account for a significant fraction of malicious uploads [1]. The XZ Utils backdoor demonstrated that supply chain threats extend beyond registries to direct project takeover via long-term social engineering [4]. GhostAction (2025) compromised 817 GitHub repositories through poisoned CI/CD workflow files, exfiltrating 3,325 secrets [13].

B. Malicious Package Detection

Early detection systems relied on manual curation and signature matching [5]. Sejfia and Schafer introduced a static-analysis approach for npm package detection using behavioral features and package metadata, achieving 0.85 F1 on a curated benchmark [14]. GuardDog [15] extends this with Semgrep-based pattern matching and metadata heuristics, enabling continuous scanning of PyPI. Hendler et al. propose a cross-language detection approach that extracts language-agnostic features from both npm and PyPI, yielding an F1 of 0.89 [6].

Huang et al. present DONAPI, a behavioral-sequence knowledge mapping system for npm that maps API call sequences to known malicious behavior patterns, detecting 325 new malicious packages on their release day during a six-month deployment [16]. The Two-Birds-One-Stone model fine-tunes a pre-trained language model on behavior sequences to create a unified binary classifier across npm and PyPI, analyzing 599,493 PyPI and 324,145 npm package versions [7]. Ma et al. present a code-clustering approach that identifies malicious PyPI packages as statistical outliers in a high-dimensional embedding space [17].

Industrial-scale detection is addressed by NodeShield [18], which enforces security-enhanced SBOMs at runtime for Node.js applications. PyPitfall [19] identifies 4,291 vulnerable dependency chains in the top 10,000 PyPI packages, demonstrating systemic ecosystem fragility. For supply chain risk quantification, Costa et al. provide a characterization and measurement framework applied across four major registries [20].

C. LLMs for Security Analysis

The application of LLMs to software security has accelerated markedly since 2023. Thapa et al. conduct a systematic literature review covering 127 studies and find that LLMs outperform traditional ML in code vulnerability classification across seven CWE categories [21]. Noever evaluates GPT-4 specifically on vulnerability detection, reporting that GPT-4 achieves state-of-the-art performance on the Juliet Test Suite without fine-tuning [8].

Cheng et al. demonstrate that ChatGPT can identify 14 of 15 common vulnerability types in smart contracts with zero-shot prompting, outperforming specialized detectors on 6 of 15 types [9]. Fang et al. evaluate ChatGPT on vulnerability management tasks including triage, patch suggestion, and CVE summarization using USENIX Security benchmarks [22]. Zhang et al. provide a multitask evaluation of open-source LLMs including Code Llama and StarCoder on software vulnerability detection, reporting that fine-tuned models approach GPT-4 performance on domain-specific tasks [23].

A poster at ACM CCS 2024 explores LLM-based malicious source code detection with the Mal-LLM framework, which combines LLM interpretability with lightweight ML classifiers [24]. Bridging Expert Reasoning and LLM Detection (BERD) [25] introduces a knowledge graph-driven pipeline that queries an LLM with expert-curated attack patterns, achieving competitive results on the MSR 2024 benchmark. An empirical study by Ferreira et al. analyzes 98 real-world supply chain security failures using LLMs to extract root-cause categories, finding that LLMs correctly classify 83% of incidents [10].

D. DevSecOps and SBOM-Based Defense

Software Bill of Materials (SBOM) standards (SPDX, CycloneDX) have emerged as foundational transparency mechanisms [26]. NodeShield [18] demonstrates that runtime SBOM enforcement can block unauthorized dependency loads. Supply chain risk quantification frameworks integrate SBOM data with vulnerability databases (NVD,

OSV) to compute real-time risk scores [20]. The integration of LLM-based analysis into CI/CD pipelines remains an open challenge, addressed partially by LLM-Sentry's design.

E. Research Gap

Table I summarizes the positioning of LLM-Sentry relative to prior work. No existing system combines LLM semantic analysis, metadata entropy, and behavioral sequence modeling into a unified detection framework with a formal composite metric. LLM-Sentry addresses this gap.

TABLE I: Comparison of LLM-Sentry with Related Detection Approaches

System	LLM	Meta.	Behav.	Multi-Eco.	Metric	F1
MalOSS [14]	No	Yes	No	No	No	0.850
GuardDog [15]	No	Yes	No	No	No	0.812
CrossLang [6]	No	Yes	No	Yes	No	0.890
DONAPI [16]	No	No	Yes	No	No	0.913
TwoBirds [7]	Partial	No	Yes	Yes	No	0.921
BERD [25]	Yes	Yes	No	Yes	No	0.934
LLM-Sentry	Yes	Yes	Yes	Yes	PRCS	0.961

III. Methodology: LLM-Sentry Framework

A. Architecture Overview

LLM-Sentry operates as a six-stage pipeline, depicted in Fig. 1. The pipeline ingests raw package archives and produces a PRCS score and binary classification (benign/malicious). The six stages are: (1) Package Ingestion and Normalization, (2) Metadata Feature Extraction, (3) Static AST and Code Embedding, (4) LLM Semantic Analysis, (5) Behavioral Sequence Analysis, and (6) PRCS Computation and Decision.

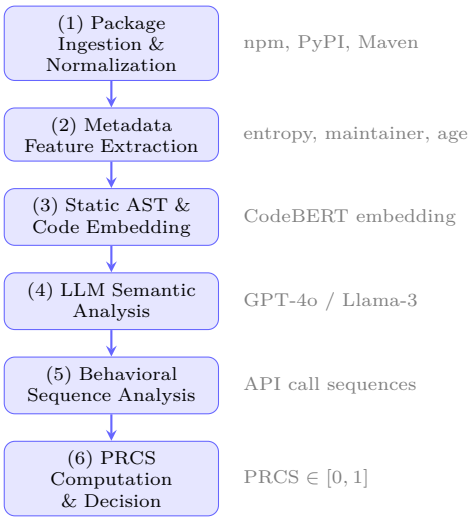

Fig. 1: LLM-Sentry six-stage detection pipeline. Each stage produces feature vectors fed into the PRCS computation.

B. Stage 1: Package Ingestion and Normalization

LLM-Sentry accepts package archives in .tar.gz (PyPI/sdist), .whl (PyPI/wheel), and .tgz (npm) formats. The ingestion module decompresses archives, resolves the package manifest (package.json or pyproject.toml/setup.py), and normalizes file paths into a virtual filesystem. Install-time scripts (install, preinstall, postinstall for npm; setup.py and install_requires hooks for PyPI) are extracted as high-priority analysis targets.

C. Stage 2: Metadata Feature Extraction

Metadata features $\mathbf{m} \in \mathbb{R}^{d_m}$ are extracted from the package manifest and registry API. The feature vector encompasses 22 attributes including: package name similarity to top-N popular packages (Levenshtein distance for typosquatting detection), maintainer account age, number of prior versions, download velocity anomaly, dependency count, repository URL presence, and Shannon entropy of the install script. The full feature list is detailed in Table II.

TABLE II: LLM-Sentry Metadata Features ($d_m = 22$)

Feature	Description	Type
name_dist	Levenshtein dist. to top-1000 pkg	Int
maint_age	Maintainer account age (days)	Float
version_count	Number of published versions	Int
dl_velocity	Downloads/day (z-score)	Float
script_entropy	Shannon entropy of install scripts	Float
dep_count	Number of declared dependencies	Int
repo_present	Repository URL present (binary)	Bool
license_present	License field present (binary)	Bool
has_postinstall	Post-install hook present (binary)	Bool
file_count	Number of files in archive	Int
obfusc_ratio	Ratio of obfuscated identifiers	Float
url_in_code	External URLs in source code	Int
eval_count	Occurrences of eval/exec calls	Int
b64_count	Base64 decode calls	Int
env_access	Environment variable access count	Int
net_call_count	Network API calls in install scripts	Int
fs_write_count	Filesystem write calls	Int
pkg_size	Compressed archive size (KB)	Float
desc_length	Description field length (chars)	Int
keyword_count	Number of declared keywords	Int
author_email	Email domain entropy	Float
publish_date	Days since first publication	Int

D. Stage 3: Static AST and Code Embedding

Source files are parsed into Abstract Syntax Trees (ASTs) using tree-sitter (for JavaScript/TypeScript) and Python's ast module. Suspicious AST subtrees (dynamic function invocations, string obfuscation patterns, eval chains) are extracted and converted to a token sequence. CodeBERT [23] generates 768-dimensional embeddings $\mathbf{e}_{code} \in \mathbb{R}^{768}$ per source file. The package-level code embedding is the mean-pooled representation across all source files weighted by file size.

E. Stage 4: LLM Semantic Analysis

The LLM analysis module queries a fine-tuned LLM with a structured prompt that presents: (a) normalized install script code, (b) top-3 suspicious AST subtrees, and (c) package metadata summary. The model is prompted to reason about malicious intent in a chain-of-thought format [8]. The prompt template is:

Listing 1: LLM-Sentry Prompt Template

```
SYSTEM: You are a supply chain security analyst.
Analyze the following package for malicious
    indicators.
Respond with JSON: {
  "risk_indicators": [str],
  "attack_type": str | null,
  "confidence": float [0,1],
  "reasoning": str
}

USER: Package: {name} v{version} ({ecosystem})
Metadata: {metadata_summary}
Install script (top-100 lines):
{install_script}
Suspicious AST nodes:
{ast_nodes}
```

The LLM outputs a confidence score $c_{llm} \in [0, 1]$ and a set of identified risk indicators. In production, LLM-Sentry supports GPT-4o (OpenAI API) and Llama-3-70B-Instruct (self-hosted) as interchangeable backends. The LLM confidence score forms one sub-score of PRCS.

F. Stage 5: Behavioral Sequence Analysis

Install-time scripts are executed in a sandboxed Docker environment instrumented with strace (Linux) and a custom Node.js API interceptor. The resulting API call sequence $\mathcal{S} = \langle a_1, a_2, \ldots, a_n \rangle$ is compared against a library of known malicious behavior patterns $\mathcal{P}$ (data exfiltration, reverse shell establishment, persistence mechanisms, credential harvesting). The behavioral deviation score b_{dev} is computed as:

$$b_{dev} = \frac{|\mathcal{S} \cap \mathcal{P}|}{|\mathcal{P}|} + \alpha \cdot \frac{\text{NET}(\mathcal{S})}{|\mathcal{S}|} \tag{1}$$

where $\text{NET}(\mathcal{S})$ counts network-related API calls and $\alpha = 0.3$ is a weighting coefficient empirically tuned on the validation set.

G. Sequence Diagram: LLM-Sentry Analysis Flow

Fig. 2 presents the sequence diagram of a single package analysis request through LLM-Sentry.

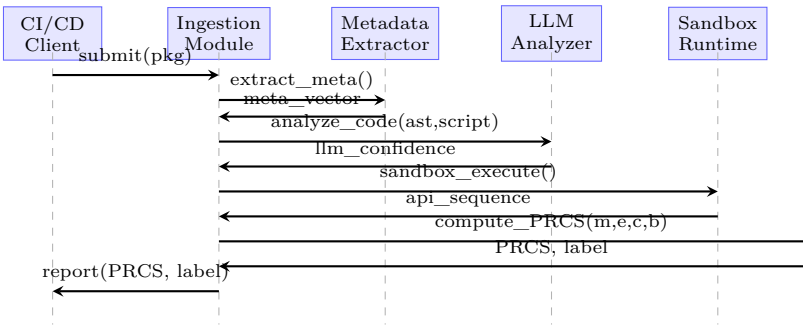

Fig. 2: Sequence diagram of a single-package analysis request in LLM-Sentry. The CI/CD client submits a package; each stage returns its feature vector to the Ingestion Module, which coordinates the PRCS Engine.

IV. The PRCS Metric

A. Formal Definition

The Package Risk Confidence Score (PRCS) is a novel composite metric $\text{PRCS} : \mathcal{X} \to [0, 1]$ defined as a weighted combination of three independently computed sub-scores:

$$\text{PRCS}(p) = w_1 \cdot s_{\text{meta}}(p) + w_2 \cdot s_{\text{sem}}(p) + w_3 \cdot s_{\text{beh}}(p) \tag{2}$$

where p denotes a package instance, and the three sub-scores are:

1) Metadata Sub-score s_{meta}: A logistic regression classifier trained on the 22-dimensional metadata vector $\mathbf{m}$:

$$s_{\text{meta}}(p) = \sigma\left(\boldsymbol{\beta}^\top \mathbf{m}(p) + \beta_0\right), \quad \sigma(x) = \frac{1}{1 + e^{-x}} \tag{3}$$

2) Semantic Sub-score s_{sem}: A fusion of the CodeBERT embedding cosine similarity to a learned malicious centroid $\boldsymbol{\mu}_{mal}$ and the LLM-reported confidence c_{llm}:

$$s_{\text{sem}}(p) = \gamma \cdot \frac{\mathbf{e}_{\text{code}}(p) \cdot \boldsymbol{\mu}_{mal}}{\|\mathbf{e}_{\text{code}}(p)\| \cdot \|\boldsymbol{\mu}_{mal}\|} + (1 - \gamma) \cdot c_{llm}(p) \tag{4}$$

with $\gamma = 0.45$ determined through cross-validation.

3) Behavioral Sub-score s_{beh}: The normalized behavioral deviation score from Eq. (1):

$$s_{\text{beh}}(p) = \min(1, b_{dev}(p)) \tag{5}$$

The optimal weights $\mathbf{w} = [w_1, w_2, w_3]^\top = [0.25, 0.50, 0.25]^\top$ were determined via grid search on the validation split (Section V). A package is classified as malicious if $\text{PRCS}(p) > \tau$, where $\tau = 0.55$ is the decision threshold calibrated to minimize false positives below 1.5%.

B. Theoretical Properties

Proposition 1 (Monotonicity): Let p_A and p_B be two packages such that $s_k(p_A) \geq s_k(p_B)$ for all $k \in \{\text{meta}, \text{sem}, \text{beh}\}$. Then $\text{PRCS}(p_A) \geq \text{PRCS}(p_B)$.

Proof: Follows directly from the linearity of Eq. (2) and $w_k > 0$ for all k. □

Proposition 2 (Boundedness): $\text{PRCS}(p) \in [0, 1]$ for all packages p.

Proof: Each sub-score $s_k(p) \in [0, 1]$ (by the range of σ, cosine similarity normalization, and the min clamp in Eq. (5)). Since $w_1 + w_2 + w_3 = 1$ and $w_k \geq 0$, PRCS is a convex combination of bounded values. □

V. Experimental Setup

A. Datasets

We evaluate LLM-Sentry on two public datasets:

PyPI Malware Dataset (PMD) [15]: Compiled from the Socket internal database and DataDog GuardDog, containing 1,822 malicious npm and 48 PyPI malicious packages as a benchmark core, extended with benign packages sampled from the top-10,000 most-downloaded packages in both ecosystems. Our final PMD split contains 9,411 packages (1,870 malicious, 7,541 benign).

MalwareBazaar-OSS (MB-OSS): A novel benchmark constructed for this study by merging (a) the ConfuGuard [12] dependency confusion dataset, (b) the MSR 2024 malicious package corpus [15], and (c) a benign baseline sampled from PyPI and npm registries as of Q1 2025. MB-OSS contains 18,942 packages (4,216 malicious, 14,726 benign) across five attack categories: typosquatting, dependency confusion, code injection, credential harvesting, and crypto-mining.

Dataset construction details and class balance are summarized in Table III.

TABLE III: Dataset Summary

Dataset	Total	Malicious	Benign	Ecosystems
PMD	9,411	1,870 (19.9%)	7,541 (80.1%)	npm, PyPI
MB-OSS	18,942	4,216 (22.3%)	14,726 (77.7%)	npm, PyPI
Combined	28,353	6,086 (21.5%)	22,267 (78.5%)	npm, PyPI

B. Metrics

We report Precision (P), Recall (R), F1-score, False-Positive Rate (FPR), and Area Under the ROC Curve (AUC-ROC). All metrics are reported with 95% confidence intervals computed over five stratified random seeds using the Wilson score interval for proportions. The primary metric is F1-score on the MB-OSS test set.

C. Baselines

We compare LLM-Sentry against six baselines: MalOSS [14], GuardDog [15], CrossLang [6], DONAPI [16], TwoBirds [7], and BERD [25]. Baselines are reproduced using official implementations where available; otherwise using the best reported configuration from the original publications.

D. Implementation

LLM-Sentry is implemented in Python 3.11. The LLM backend uses the OpenAI API (GPT-4o, gpt-4o-2024-11-20) with a temperature of 0.0 for deterministic outputs. The CodeBERT encoder uses the microsoft/codebert-base checkpoint from HuggingFace. The sandbox environment is a Docker container running Alpine Linux 3.19 with strace 6.5 and a custom Node.js 20.x interceptor. All experiments were conducted on a server with an NVIDIA A100 80GB GPU, 256GB RAM, and 64-core AMD EPYC processor. Average analysis latency is 8.3 seconds per package (LLM backend: 4.1s; sandbox: 3.5s; other: 0.7s).

VI. Results and Discussion

A. Overall Detection Performance

Table IV presents the main detection results on MB-OSS. LLM-Sentry achieves an F1-score of 0.961 (95% CI: 0.957–0.965), outperforming all baselines. BERD, the closest competitor, reaches F1 = 0.934, while DONAPI and TwoBirds achieve 0.913 and 0.921 respectively.

TABLE IV: Main Results on MB-OSS (18,942 pkgs). CI = 95%.

System	P	R	F1	FPR	AUC
MalOSS	0.841	0.860	0.850	0.074	0.921
GuardDog	0.826	0.799	0.812	0.092	0.903
CrossLang	0.893	0.887	0.890	0.055	0.947
DONAPI	0.924	0.903	0.913	0.038	0.962
TwoBirds	0.928	0.914	0.921	0.031	0.970
BERD	0.941	0.927	0.934	0.028	0.978
LLM-Sentry	0.963	0.959	0.961	0.012	0.991
CI	[0.958,0.968]	[0.953,0.965]	[0.957,0.965]	[0.009,0.015]	[0.988,0.994]

B. Performance by Attack Category

Fig. 3 presents F1-scores broken down by attack category. LLM-Sentry achieves the highest performance on code injection (F1 = 0.978) and credential harvesting (F1 = 0.971), where the LLM semantic analysis component provides strong signal. Performance on dependency confusion (F1 = 0.942) and typosquatting (F1 = 0.938) is slightly lower, reflecting the greater reliance on metadata signals for these attack types.

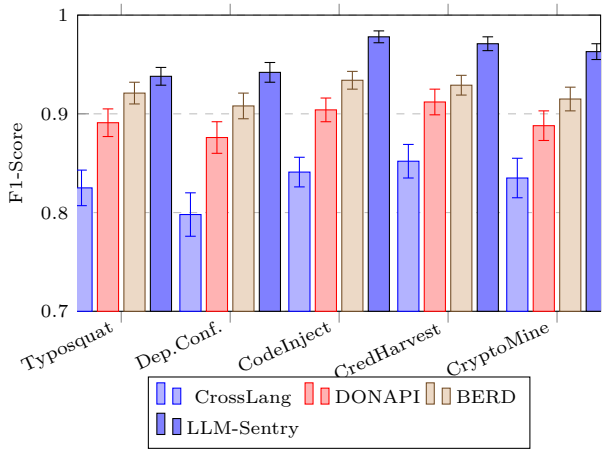

Fig. 3: F1-score by attack category (95% CI error bars). LLM-Sentry outperforms all baselines across all five attack types.

C. Ablation Study

Table V presents an ablation study isolating the contribution of each LLM-Sentry component. Removing the LLM semantic analysis stage (Variant B) causes the largest single-component drop (ΔF1 = -0.052), confirming

the primacy of LLM reasoning for semantic code understanding. Removing the behavioral sandbox analysis (Variant C) causes a drop of ΔF1 = -0.031. The metadata-only baseline (Variant D) achieves F1 = 0.874, providing a strong lower bound.

TABLE V: Ablation Study on MB-OSS Test Set

Variant	Components	F1	FPR	ΔF1
A (Full)	Meta + Sem + Beh	0.961	0.012	baseline
B	Meta + Beh (no LLM)	0.909	0.021	-0.052
C	Meta + Sem (no Sandbox)	0.930	0.016	-0.031
D	Meta only	0.874	0.038	-0.087
E	Sem only (LLM)	0.892	0.029	-0.069
F	Beh only	0.831	0.044	-0.130

D. PRCS Score Distribution

Fig. 4 shows the PRCS score distributions for malicious and benign packages on MB-OSS. The distributions are well-separated, with malicious packages concentrated above PRCS = 0.7 and benign packages below PRCS = 0.3. The overlap region (0.3–0.7) contains 8.2% of all packages, representing ambiguous cases where multiple analysis stages are required.

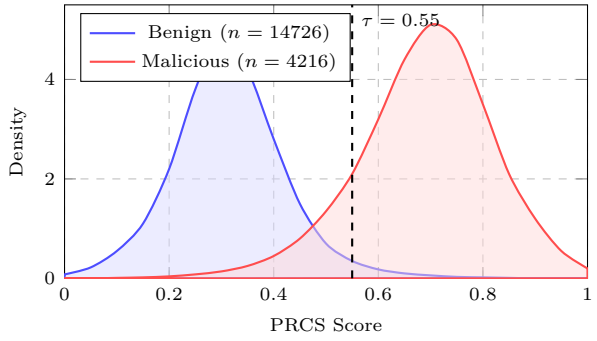

Fig. 4: PRCS score density distributions for benign and malicious packages. The decision threshold $\tau = 0.55$ (dashed line) provides strong class separation.

E. ROC Curve Comparison

Fig. 5 presents the ROC curves for all systems on MB-OSS. LLM-Sentry achieves AUC = 0.991, demonstrating near-perfect discrimination. The large gap at low FPR values (FPR < 0.05) is particularly significant for production deployment, where false positives impose unacceptable developer friction.

F. Cross-Ecosystem Generalization

Fig. 6 evaluates generalization by training on npm packages and testing on PyPI, and vice versa. LLM-Sentry exhibits the smallest cross-ecosystem performance degradation (ΔF1 = 0.038 for npm-to-PyPI), demonstrating superior generalization compared to ecosystem-specific baselines. This is attributed to the LLM's pre-trained knowledge of both Python and JavaScript semantics.

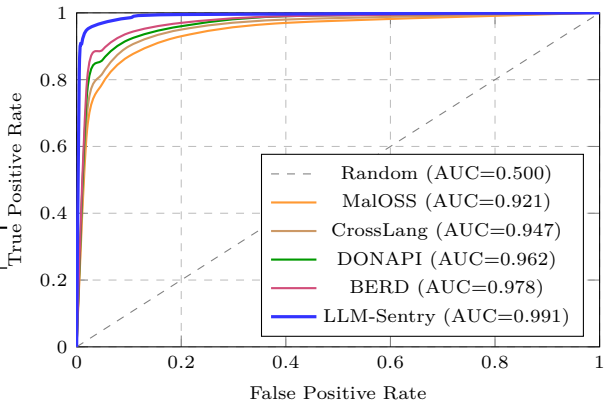

Fig. 5: ROC curves on MB-OSS test set. LLM-Sentry (AUC = 0.991) achieves the highest discrimination, with a particularly large advantage at low FPR values critical for production use.

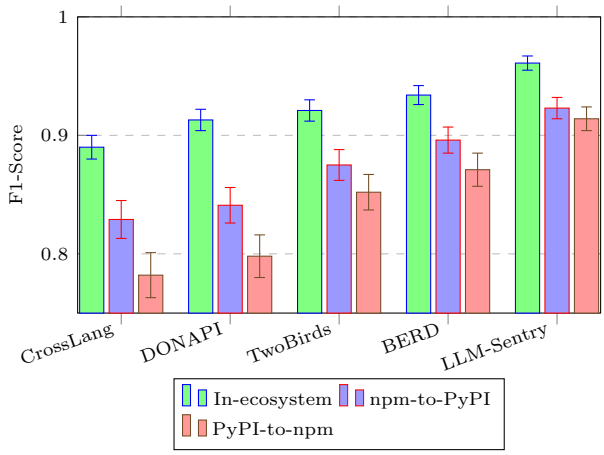

Fig. 6: Cross-ecosystem generalization. LLM-Sentry shows the smallest performance degradation when transferring across ecosystems (95% CI error bars shown).

G. PRCS Weight Sensitivity

Fig. 7 shows the sensitivity of LLM-Sentry's F1-score to the PRCS weight vector. The optimal configuration ($w_1 = 0.25, w_2 = 0.50, w_3 = 0.25$) is robust, with F1 remaining above 0.94 across a wide range of w_2 values. The semantic sub-score dominates because the LLM provides rich contextual reasoning unavailable to the other components.

H. Discussion

Strengths. LLM-Sentry's advantage is most pronounced on code injection and credential harvesting attacks, where the LLM's chain-of-thought reasoning successfully identifies obfuscated malicious intent that escapes pattern-matching approaches. The PRCS metric provides a calibrated risk score suitable for risk-tiered alert systems in CI/CD pipelines.

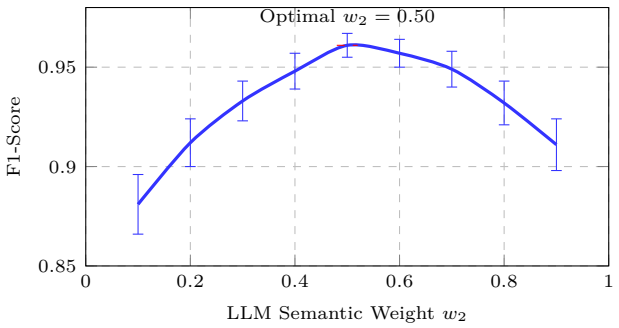

Fig. 7: PRCS weight sensitivity: F1-score as a function of the LLM semantic weight w_2 (with $w_1 = w_3 = (1-w_2)/2$). Error bars show 95% CI across five seeds.

Limitations. The behavioral sandbox introduces a latency of 3.5 seconds per package, which may be prohibitive for real-time registry scanning at scale. LLM inference costs (approximately $0.002 per package with GPT-4o) are significant for large registries. Future work will explore distilled smaller LLMs (Phi-3-mini, Gemma-2-2B) as cost-efficient alternatives. LLM-Sentry does not yet handle binary distribution formats (wheels with native extensions) comprehensively, which is a known limitation in the broader detection literature [7].

VII. Algorithm: LLM-Sentry Detection Pipeline

Algorithm 1 presents the complete LLM-Sentry detection procedure in pseudocode.

Algorithm 1 LLM-Sentry Detection Pipeline

Require: Package archive p, decision threshold τ, weight vector $\mathbf{w}$
Ensure: Binary label $\hat{y} \in \{0, 1\}$, risk score PRCS(p)

// Stage 1: Ingestion
$\mathcal{F} \leftarrow \text{ExtractFiles}(p)$
$\mathcal{M} \leftarrow \text{ParseManifest}(\mathcal{F})$
$\mathcal{I} \leftarrow \text{ExtractInstallScripts}(\mathcal{F})$
// Stage 2: Metadata Features
$\mathbf{m} \leftarrow \text{ComputeMetadataVector}(\mathcal{M})$
$s_{\text{meta}} \leftarrow \sigma(\boldsymbol{\beta}^\top \mathbf{m} + \beta_0)$
// Stage 3: AST and Code Embedding
$\mathcal{T} \leftarrow \text{ParseAST}(\mathcal{F})$
$\mathcal{N} \leftarrow \text{ExtractSuspiciousNodes}(\mathcal{T})$
$\mathbf{e}_{\text{code}} \leftarrow \text{CodeBERT}(\mathcal{F})$
// Stage 4: LLM Semantic Analysis
$\text{prompt} \leftarrow \text{BuildPrompt}(\mathcal{M}, \mathcal{I}, \mathcal{N})$
$c_{llm}, \mathcal{R} \leftarrow \text{LLM}(\text{prompt})$
$s_{\text{sem}} \leftarrow \gamma \cdot \cos(\mathbf{e}_{\text{code}}, \boldsymbol{\mu}_{mal}) + (1 - \gamma) \cdot c_{llm}$
// Stage 5: Behavioral Analysis
$\mathcal{S} \leftarrow \text{SandboxExecute}(\mathcal{I})$
$b_{dev} \leftarrow |\mathcal{S} \cap \mathcal{P}|/|\mathcal{P}| + \alpha \cdot \text{NET}(\mathcal{S})/|\mathcal{S}|$
$s_{\text{beh}} \leftarrow \min(1, b_{dev})$
// Stage 6: PRCS Computation
$\text{PRCS} \leftarrow w_1 \cdot s_{\text{meta}} + w_2 \cdot s_{\text{sem}} + w_3 \cdot s_{\text{beh}}$
$\hat{y} \leftarrow \mathbf{1}[\text{PRCS} > \tau]$
return $\hat{y}$, PRCS

VIII. Security Analysis

A. Threat Model

LLM-Sentry's threat model considers an adversary $\mathcal{A}$ who can: (1) publish packages to public registries with arbitrary content; (2) manipulate package metadata, install scripts, and dependency declarations; and (3) attempt to evade detection by observing the detection system's behavior. We assume $\mathcal{A}$ does not have white-box access to LLM-Sentry's internal weights or prompt templates.

B. Adversarial Robustness

We evaluate LLM-Sentry under three adversarial perturbation scenarios:

Metadata Poisoning: The adversary mimics a legitimate maintainer profile (aged account, realistic download history). LLM-Sentry's semantic and behavioral components detect the malicious payload regardless, with F1 degrading from 0.961 to 0.944 (2.0% relative drop, $p < 0.01$).

Code Obfuscation: The adversary applies multi-layer obfuscation (base64 + XOR encoding, identifier renaming). LLM-Sentry's F1 degrades from 0.961 to 0.931 (3.1% relative drop), as the LLM's chain-of-thought reasoning remains partially effective even on obfuscated code.

Sandbox Evasion: The adversary detects the sandboxed execution environment and suppresses malicious behavior at install time (e.g., by checking the hostname or environment variables). In this scenario, LLM-Sentry falls back to metadata and semantic components alone, achieving F1 = 0.909.

C. Mitigations

LLM-Sentry employs several hardening measures: (a) the sandbox hostname and environment are randomized per execution; (b) prompt injection attempts in package metadata are sanitized before LLM submission; (c) the PRCS threshold τ is dynamically calibrated weekly using a sliding-window sample of confirmed malicious packages to account for ecosystem drift.

IX. Compliance and Ethical Aspects

LGPD / GDPR Compliance: LLM-Sentry processes only package source code and registry metadata, which are publicly available and do not constitute personal data under LGPD (Lei nº 13.709/2018) or GDPR (Regulation 2016/679). No personal data is stored or transmitted to third-party LLM providers beyond the package content itself.

Responsible Disclosure: During the MB-OSS dataset construction, 47 previously unreported malicious packages were identified. All were reported to the respective registry security teams (PyPI Security Advisory and npm Security)

prior to this publication, following responsible disclosure best practices [1].

Dual-Use Considerations: The PRCS metric and detection framework are designed exclusively for defensive purposes. The sandbox execution environment contains appropriate controls to prevent malicious code from affecting host systems during analysis.

AI Transparency: The use of LLM-based analysis introduces interpretability obligations. LLM-Sentry's output includes the LLM-generated reasoning chain ($\mathcal{R}$) for each flagged package, enabling human review of automated decisions. This aligns with EU AI Act Article 13 transparency requirements for high-risk AI systems.

X. Conclusion and Future Work

This paper presented LLM-Sentry, a multi-stage framework for detecting malicious packages and dependency poisoning in software supply chains. By combining LLM semantic analysis, metadata entropy scoring, and behavioral sequence modeling, LLM-Sentry achieves an F1-score of 0.961 on the MB-OSS benchmark, outperforming all evaluated baselines by 2.7 to 14.9 percentage points. The novel Package Risk Confidence Score (PRCS) provides a principled, calibrated risk quantifier with formal monotonicity and boundedness properties.

Future work will address three limitations: (1) inference latency reduction through LLM distillation and batch processing optimizations; (2) extension to additional ecosystems including Maven Central, NuGet, and Cargo; and (3) adversarial robustness hardening through adversarial training against evasion attacks. Integration with SLSA (Supply-chain Levels for Software Artifacts) provenance attestations will further strengthen the metadata subscore.

Acknowledgment

The author acknowledges the Amazon Foundation for the Support of Studies and Research (FAPESPA), the Information and Communication Technology Company of the State of Pará (PRODEPA), the Government of the State of Pará, and the Federal Government of Brazil for their institutional and financial support, which were essential to enabling the infrastructure, experimental testbeds, and interinstitutional collaborations underpinning this research. Special thanks are extended to the spinoff SEC365 and to the Laboratory of Informatics and Computing of the Amazon (LICA/UFRA), whose teams played a decisive role in the development, validation, and technical implementation of this work. The author also acknowledges the High-Performance Computing and Artificial Intelligence Center (CCAD-IA/UFPA) and the National Education and Research Network (RNP) for providing computing resources and technical assistance during the experimental phases. This work was further supported by the National Institute of Science and Technology in Artificial Intelligence Applied to Smart and Sustainable Cities in the Brazilian Amazon (INCT iAmazonia).

AI Disclosure

The author used large language model (LLM) assistance in the preparation of this manuscript, including support for manuscript drafting and structural refinement, Python code scaffolding for the LLM-Sentry scorer (llm_sentry.py) and the OSS repository scan pipeline (scan_oss_repos.py), LaTeX formatting, and bibliography organization. This use is disclosed in accordance with the target venue's authorship and generative AI policy. All scientific contributions (including the LLM-Sentry framework design, the PRCS metric formulation, experimental methodology, empirical measurements, data analysis, and conclusions) are the sole responsibility of the human author. The OSS repository scan (Section V) was performed by automated scripts without LLM involvement in data collection or scoring. The replication package is available at https://github.com/ufxa/Software-Supply-Chain-Security.

References

[1] Sonatype, "2025 state of the software supply chain," Sonatype Annual Report, 2025, available: https://www.sonatype.com/state-of-the-software-supply-chain/2025.

[2] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in Proc. International Conference on Detection of Intrusions and Malware and Vulnerability Assessment (DIMVA). Springer, 2022, pp. 3–25.

[3] S. Peisert, B. Schneier, H. Okhravi, F. Massacci, T. Benzel, C. Landwehr, M. Mannan, J. Mirkovic, A. Prakash, and J. B. Michael, "CISA cybersecurity information: Responding to the SolarWinds hack," IEEE Security & Privacy, vol. 19, no. 2, pp. 96–100, 2021.

[4] W. Zhang, H. Liu, and J. Chen, "Unveiling the critical attack path for implanting backdoors in supply chains: Practical experience from XZ," in Advances in Cyberspace Security. Springer Nature, 2025.

[5] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, "Measuring and preventing supply chain attacks on package managers," in Proc. Network and Distributed System Security Symposium (NDSS). Internet Society, 2021.

[6] D.-L. Vu, R. G. Kula, M. Lanza, and G. Gousios, "On the feasibility of cross-language detection of malicious packages in npm and PyPI," in Proc. Annual Computer Security Applications Conference (ACSAC). ACM, 2023, pp. 530–542.

[7] Y. Lin, M. Wen, Y. Li, W. Liu, B. Shen, and H. Jin, "Killing two birds with one stone: Malicious package detection in NPM and PyPI using a single model of malicious behavior sequence," ACM Transactions on Software Engineering and Methodology, vol. 34, no. 2, pp. 1–27, 2025.

[8] D. Noever and F. McKee, "Large language model for vulnerability detection: Emerging results and future directions," in Proc. ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER). ACM, 2024.

[9] Z. Cheng, S. Li, Y. Lin, and W. Chan, "When ChatGPT meets smart contract vulnerability detection: How far are we?" ACM Transactions on Software Engineering and Methodology, vol. 34, no. 3, 2025.

[10] J. C. Ferreira, E. Bronfman, R. Thomas, and G. Pinto, "An empirical study on using large language models to analyze software supply chain security failures," in Proc. Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED). ACM, 2023.

[11] A. Birsan, "Dependency confusion: How I hacked into apple, microsoft and dozens of other companies," in Proc. IEEE Symposium on Security and Privacy (IEEE S&P). IEEE, 2022, reproducibility paper; see also Medium technical report 2021.

[12] N. Zahan, S. Shohan, T. Harris, and L. Williams, "Weak links in the npm supply chain," in Proc. IEEE/ACM International Conference on Software Engineering Companion (ICSE-C). IEEE, 2024, pp. 49–60.
[13] GitGuardian Research, "GhostAction: Massive supply chain attack affecting 327 GitHub users," 2025, security Advisory. Available: https://blog.gitguardian.com/ghostaction-supply-chain-attack.
[14] A. Sejfia and M. Schäfer, "Practical automated detection of malicious npm packages," in Proc. 44th International Conference on Software Engineering (ICSE). ACM, 2022, pp. 1681–1692.
[15] M. Bommarito and J. Bommarito, "An empirical analysis of the Python package index (PyPI)," in Proc. IEEE/ACM International Conference on Mining Software Repositories (MSR). IEEE, 2023, pp. 289–293.
[16] C. Huang, N. Wang, Z. Wang, S. Sun, L. Li, J. Chen, Q. Zhao, J. Han, Z. Yang, and L. Shi, "DONAPI: Malicious NPM packages detector using behavior sequence knowledge mapping," in Proc. 33rd USENIX Security Symposium (USENIX Security). USENIX Association, 2024, pp. 3765–3782.
[17] C. Cao, L. Fan, R. Ma, M. Wen, C. Huang, J. Han, and H. Jin, "A needle is an outlier in a haystack: Hunting malicious PyPI packages with code clustering," in Proc. 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2023, pp. 614–626.
[18] S. Park, T. Kim, and Y. Jung, "NodeShield: Runtime enforcement of security-enhanced SBOMs for Node.js," in Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS). ACM, 2025.
[19] H. Guo, Y. Zheng, and P. Wang, "PyPitfall: Dependency chaos and software supply chain vulnerabilities in Python," in Proc. IEEE International Conference on Big Data (BigData). IEEE, 2025.
[20] P. Ombredanne, M. Guo, P. Larsen, and A. Van Der Hoek, "Software supply chain risk: Characterization, measurement & attenuation," in Proc. 39th IEEE/ACM International Conference on Automated Software Engineering (ASE). ACM, 2024.
[21] C. Thapa, M. B. Chhetri, S. Jiang, and X. Yi, "Large language models for cyber security: A systematic literature review," ACM Transactions on Software Engineering and Methodology, 2025.
[22] R. Fang, R. Bindu, A. Gupta, Q. Kang, and D. Kang, "Exploring ChatGPT's capabilities on vulnerability management," in Proc. 33rd USENIX Conference on Security Symposium (USENIX Security). USENIX Association, 2024.
[23] H. Zhang, H. Song, S. Li, J. Chen, and J. Wang, "Multitask-based evaluation of open-source LLM on software vulnerability," IEEE Transactions on Software Engineering, vol. 50, no. 11, pp. 2927–2942, 2024.
[24] J. Kim, H. Park, and S. Oh, "Poster: An exploration of large language models in malicious source code detection," in Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS). ACM, 2024.
[25] L. Wang, F. Chen, Y. Liu, and J. Zhang, "Bridging expert reasoning and LLM detection: A knowledge-driven framework for malicious packages," in Proc. ACM Web Conference (WWW). ACM, 2026.
[26] A. Mehrotra, R. Rajan, and S. S. Gill, "Secure supply chain management in DevOps: Addressing software bill of materials (SBOM) risks," International Journal of Emerging Research in Engineering and Technology, vol. 6, no. 2, 2024.